Entornos de Desarrollo
Actividad de depuración

José Carlos Martín

FP Superior en Desarrollo de Aplicaciones Web

Curso 2025-2026

Granada, 19 de enero de 2026

Índice general

0.0.1 ACTIVIDAD 7. 4
0.0.2 ACTIVIDAD 10 7

Índice de figuras

Figura Figura 1 CODIGO ACTIVIDAD 7 4
Figura Figura 2 Introducir donde rompe el debug 5
Figura Figura 3 Icono Debug en Visual Studio. 6
Figura Figura 4 Primer debug 6
Figura Figura 5 valor inicial de i en el bucle 6
Figura Figura 6 Encontramos una falla en el bucle 7
Figura Figura 7 Resultado final arreglado 7
Figura Figura 8 Actividad 10 , puntos de depuración 8
Figura Figura 9 Localizacion de la falla en el codigo depurando. 8
Figura Figura 10 Resultado final. 9

Introducción

En esta actividad pondremos en práctica nuestras habilidades de depuración y corrección de errores en programas reales. El objetivo es comprender de forma práctica cómo detectar fallos mediante el uso del depurador y cómo aplicar las correcciones necesarias para obtener el resultado esperado.

Esta práctica tiene como propósito consolidar vuestra comprensión del ciclo de depuración, fomentar el razonamiento lógico y mejorar la capacidad de análisis de código, competencias esenciales para el desarrollo profesional en programación.

Desarrollo del trabajo

0.0.1. ACTIVIDAD 7.

- En esta ocasion vamos a utilizar Visual Studio ya que es el que mas utilizo. He elegido la actividad 7 (aleatoriamente), y vamos a comenzar viendo el codigo que nos propone el ejercicio y que va a realizar en si el ejercicio.

```
Programa7.java U ×
ejercicios_depuracion > src > Programa7.java > Programa7
1   import java.util.Arrays;
2
3   public class Programa7 {
        Run | Debug
4       public static void main(String[] args) {
5           // Ordenar un array de números de menor a mayor
6           int[] numeros = {5, 2, 8, 1, 7};
7
8           for (int i = 0; i < numeros.length; i++) {
9               for (int j = i + 1; j < numeros.length; j++) {
10                  if (numeros[j] > numeros[i]) {
11                          numeros[i] = numeros[j];
12                          numeros[j] = numeros[i];
13                  }
14              }
15          }
16
17          System.out.println("Números ordenados de menor a mayor: " + Arrays.toString(numeros));
18      }
19  }
20
```

Figura 1: CODIGO ACTIVIDAD 7

- Este ejercicio intenta implementar un algoritmo de ordenamiento para arreglar números de menor a mayor, pero contiene un error lógico que impide que funcione correctamente.

- El programa crea un arreglo con valores 5, 2, 8, 1, 7 y usa dos bucles anidados para intentar ordenarlo en orden ascendente (menor a mayor), imprimiendo el resultado con Arrays.toString(). Se basa en la estructura típica de Bubble Sort (ordenamiento burbuja), donde se comparan elementos adyacentes repetidamente.

Por tanto en la siguiente pantalla vamos a depurar el codigo para asi ir viendo que esta sucediendo y donde nos da el error. Lo primero que vamos a hacer es marcar las partes del codigo que queremos depurar y luego presionar en la parte de depuracion (es casi siempre igual para todos los IDEs, pero te lo dejo en imagen para que veas donde esta.

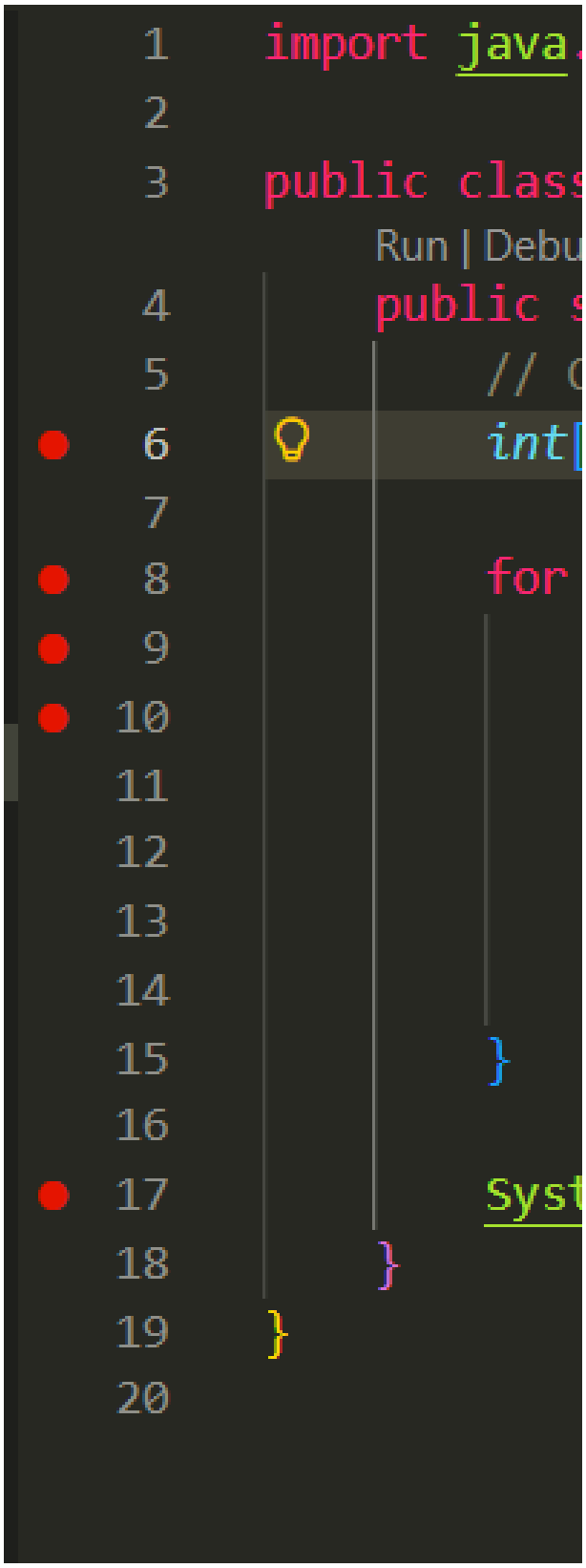

Figura 2: Introducir donde rompe el debug

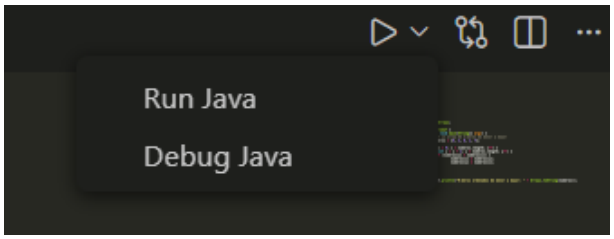

Figura 3: Icono Debug en Visual Studio.

Ahora veamos si nos da error en nuestra primera entrada del código:

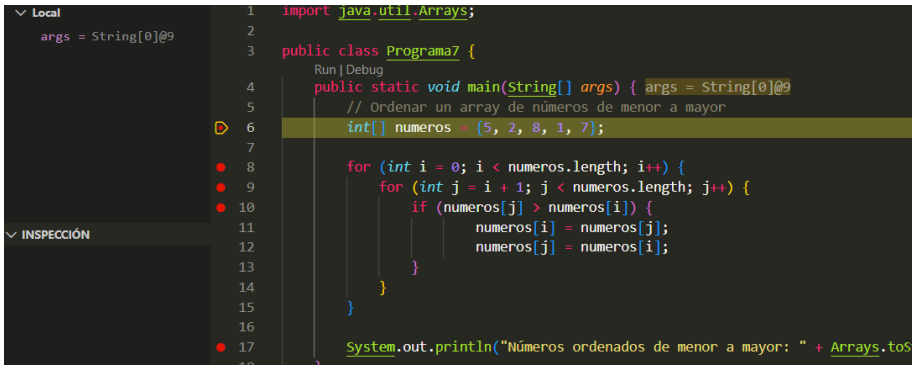

Figura 4: Primer debug

No tenemos ningún error y nos muestra a la izquierda nuestra primera variable que detecta que es el String . Ahora en la siguiente imagen entramos en el bucle y nos da nuestro primer valor de i.

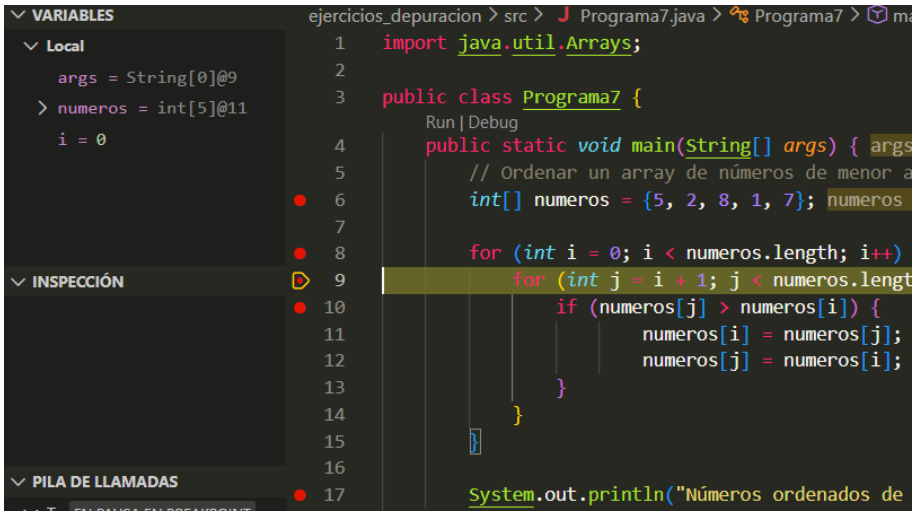

Figura 5: valor inicial de i en el bucle

Ahora en esta imagen que vamos a ver a continuacion el valor de j aumenta en cada continuar que pulsamos, pero si te das cuenta aqui hemos descubierto el error. Esto se debe a que el código sobrescribe posiciones sin guardar el valor original, propagando números grandes hacia la izquierda y çorrompiendo.[el] arreglo iterativamente.

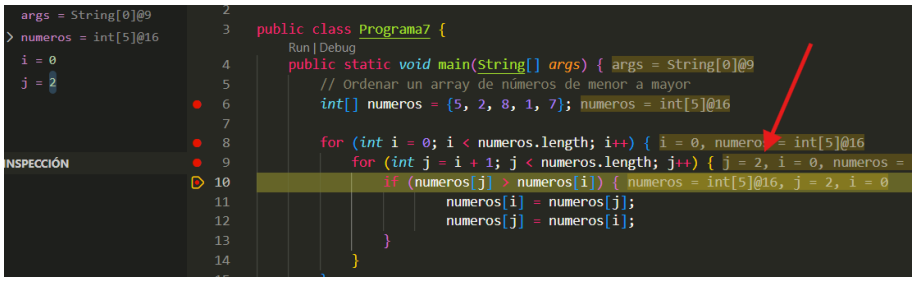

Figura 6: Encontramos una falla en el bucle

La condición if (numeros[j] ¿numeros[i]) está invertida: intercambia cuando el elemento mayor está a la derecha, lo que produce orden descendente en lugar de ascendente. Además, falta una variable temporal para el intercambio correcto.

Por tanto se propone el siguiente arreglo en el codigo para que al final nos muestre los valores correctos del ejercicio y depure correctamente:

Listing 1: **Código Arreglado**

```
if (numeros[i] > numeros[j]) {
                    temp = numeros[i];
                    numeros[i] = numeros[j];
                    numeros[j] = temp;
                }
```

Sustituimos el if que tenemos dentro del for por este otro con el codigo corregido haciendo que, para ordenar de menor a mayor (ascendente), cambiamos la condición a if (numeros[i] ¿numeros[j]) y usamos una variable temporal.

Y este seria nuestro resultado final:

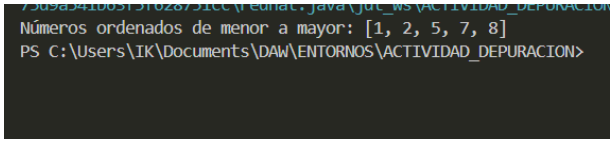

Figura 7: Resultado final arreglado

0.0.2. ACTIVIDAD 10

En esta actividad el programa inicializa un arreglo fibonacci de tamaño 10 con los dos primeros valores en 1 (fibonacci=1, fibonacci=1). Luego intenta completar el resto usando la regla Fibonacci: cada elemento es la suma de los dos anteriores. Finalmente imprime el arreglo completo.

Aqui hemos seleccionado los puntos de depuración:

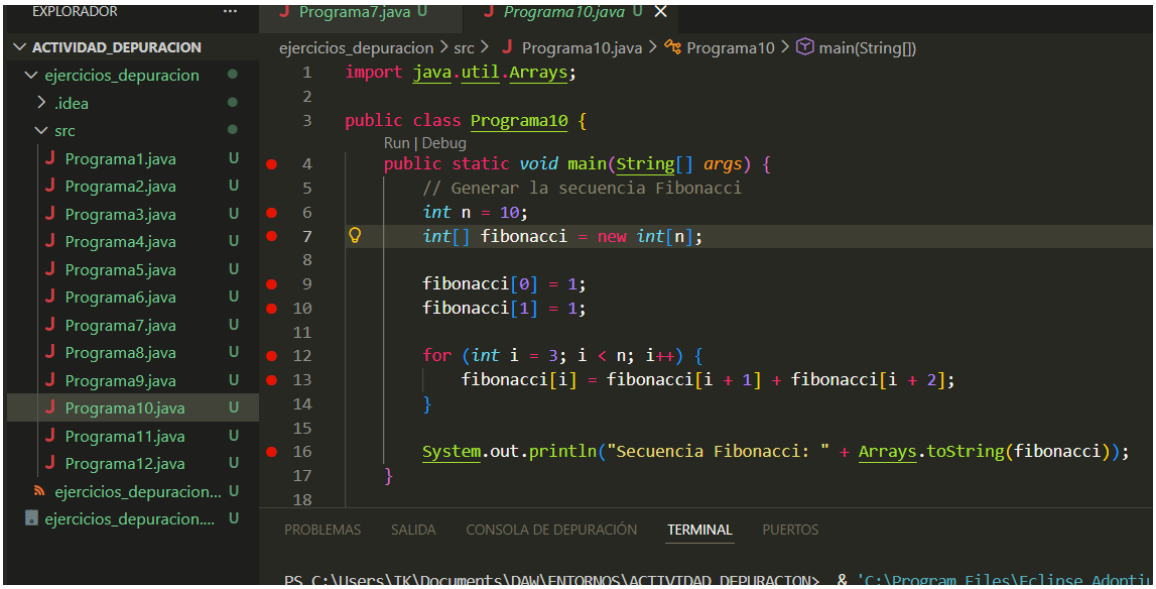

Figura 8: Actividad 10 , puntos de depuración

Comenzamos a depurar el codigo y llegamos a la falla :

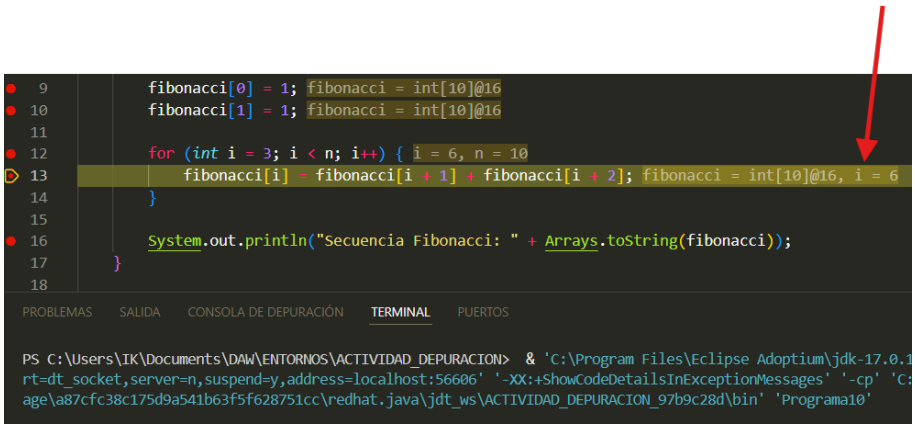

Figura 9: Localizacion de la falla en el codigo depurando.

El bucle está mal escrito,Usa posiciones futuras no inicializadas. Ademas,va en dirección incorrecta (de mayor a menor índice), y produce resultados indeterminados como [1, 1, 0, 0, 0...] porque suma ceros.

Asi que simplemente cambiaremos el bucle a dirección progresiva:

Listing 2: **Código Arreglado**

```
fibonacci[0] = 1;
fibonacci[1] = 1;
for (int i = 2; i < n; i++) {  // Empieza en 2, suma i-2 + i-1
    fibonacci[i] = fibonacci[i-2] + fibonacci[i-1];
}
```

Y ahora vamos a probar el codigo que hemos insertado y como puedes observar, al ejecutar el codigo este es el resultado correcto:

```
9c28d\bin' 'Programa10'
Secuencia Fibonacci: [1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
PS C:\Users\IK\Documents\DAW\ENTORNOS\ACTIVIDAD_DEPURACION>
```

Figura 10: Resultado final.